

Creating a Virtual Cello Music 421 Final Project

Peder Larson

June 11, 2003

1 Abstract

A virtual cello, or any other stringed instrument, can be created using digital waveguides, digital filters, and a non-linear bow-string function. The waveguides simulate both transverse and torsional wave propagation along the length of a string. Digital filters are used to model the bridge, string losses, string stiffness, and the body of the cello. A host of non-linear bow-string functions exist and can simulate bow velocity, bow pressure, bow position, and also bow width. The result of combining these pieces together is a decent sounding model of a cello.

2 Introduction

The goal throughout this project has been to create a virtual cello model that is physically accurate and produces good music. I chose the cello simply because I am a cellist and thus I feel like I can evaluate well what a cello should sound like.

Figure 1 shows a high-level waveguide description of the model. Velocity waves are used because the bow interacts based on its velocity relative to the string velocity. Both transverse and torsional waves are implemented in the final model, as well as a body filter based on an actual cello impulse response. A couple bow-string junction characteristics are also examined. Sound samples are available at <http://www-ccrma.stanford.edu/~plarson/cello.html>.

3 Cello Body

The cello body is considerably larger than the violin and viola bodies, although not as large as the bass. The size of each instrument corresponds to its pitch range, with the larger bodies having lower fundamental resonances to bring out the low pitches produced by the strings. A well designed and crafted cello body creates a full and rich tone.

A cello body impulse response and its spectrum are shown in figure 2. The Helmholtz mode - the least damped mode - is derived from the Fourier Transform of the late echo, and is about 87.8 Hz. Another mode of slightly less amplitude shows up also at 80 Hz, while the next mode shows up at 158 Hz. For the practical purpose of convolving with the impulse response, which is about 6000 samples, it is helpful to remove the Helmholtz and/or other modes to create a shortened impulse response that is computationally manageable. BiQuad filters can be used to model these removed modes.

I used the technique of localized peak removal to get rid of the a few of the modes. Figure 3 shows the the impulse response with the two lowest modes removed and the corresponding frequency response. The impulse response is considerably shortened by a factor of 3, to a length of about 2000 samples. This impulse response creates an FIR filter that is used in series with two BiQuad filters at the output of the strings to simulate the resonance of the cello body. Adding these filters into the model creates a significant improvement in sound and tone quality.

4 String Model

The string model is derived from the differential equation for the ideal vibrating string:

$$Ky'' = \epsilon \ddot{y} \tag{1}$$

where K is the string tension, ϵ is the linear mass density, and y is the string displacement. It simply corresponds to delay lines with a total round trip length of N samples, computed as $N = f_s / f_0$ where f_s is the sampling rate and f_0 is the frequency of the string. In figure 1, $N = 2N_1 + 2N + 2$.

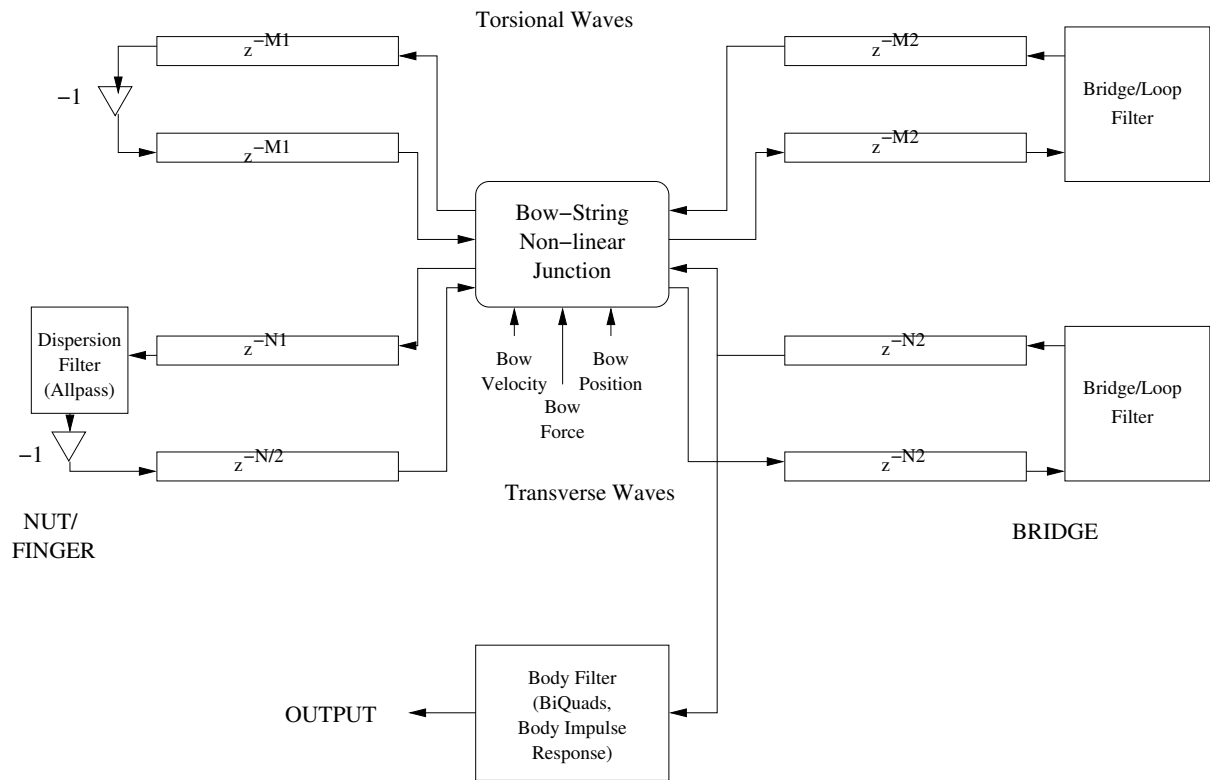


Figure 1: This is the complete digital waveguide model for the cello. In the actual implementation, delays/filters/gains are combined wherever possible. Also note that the location of the output tap is arbitrary.

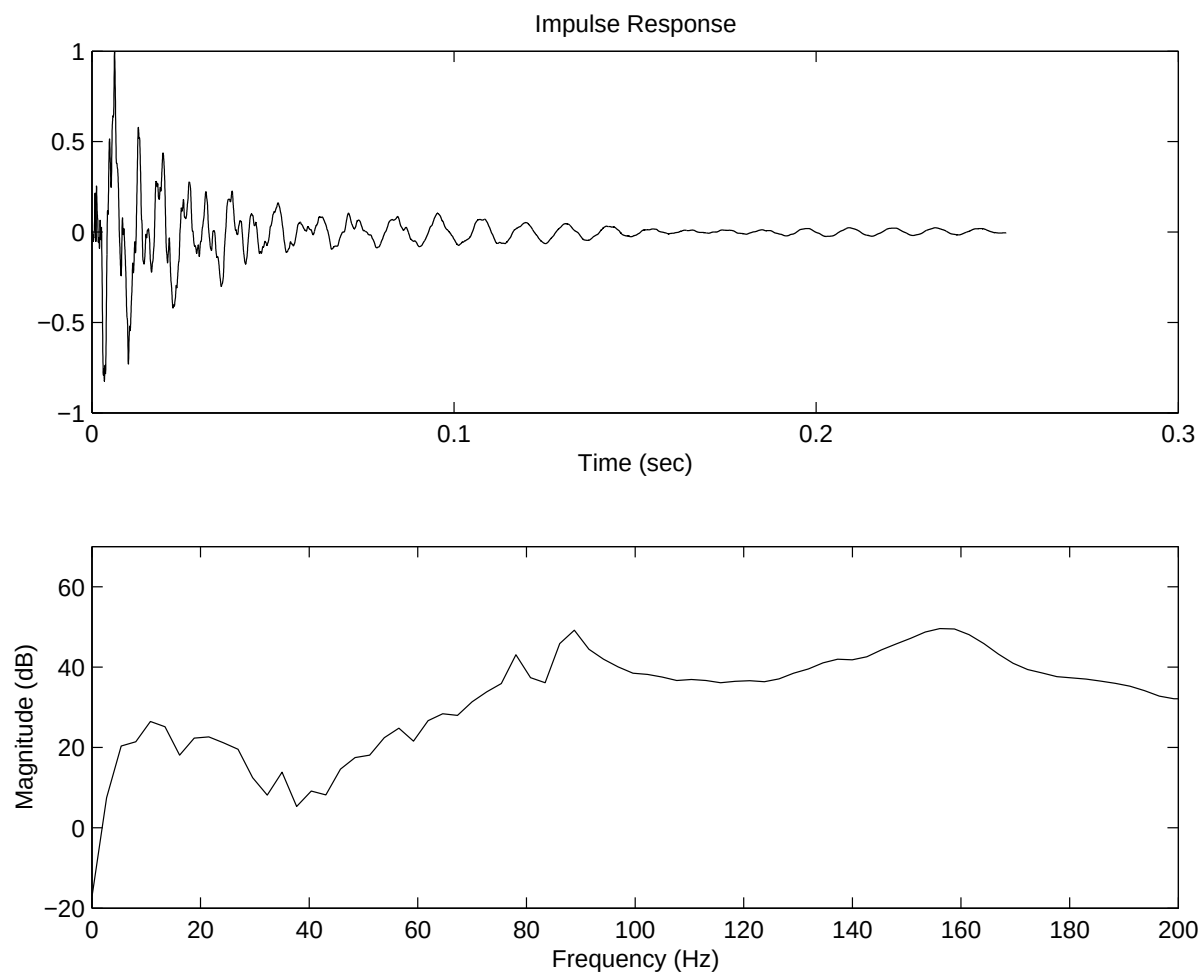


Figure 2: Shown are the impulse response and low-frequency transfer function of a cello body impulse response. The Helmholtz mode is visible in the impulse response, and is the peak near 87 Hz in the magnitude response.

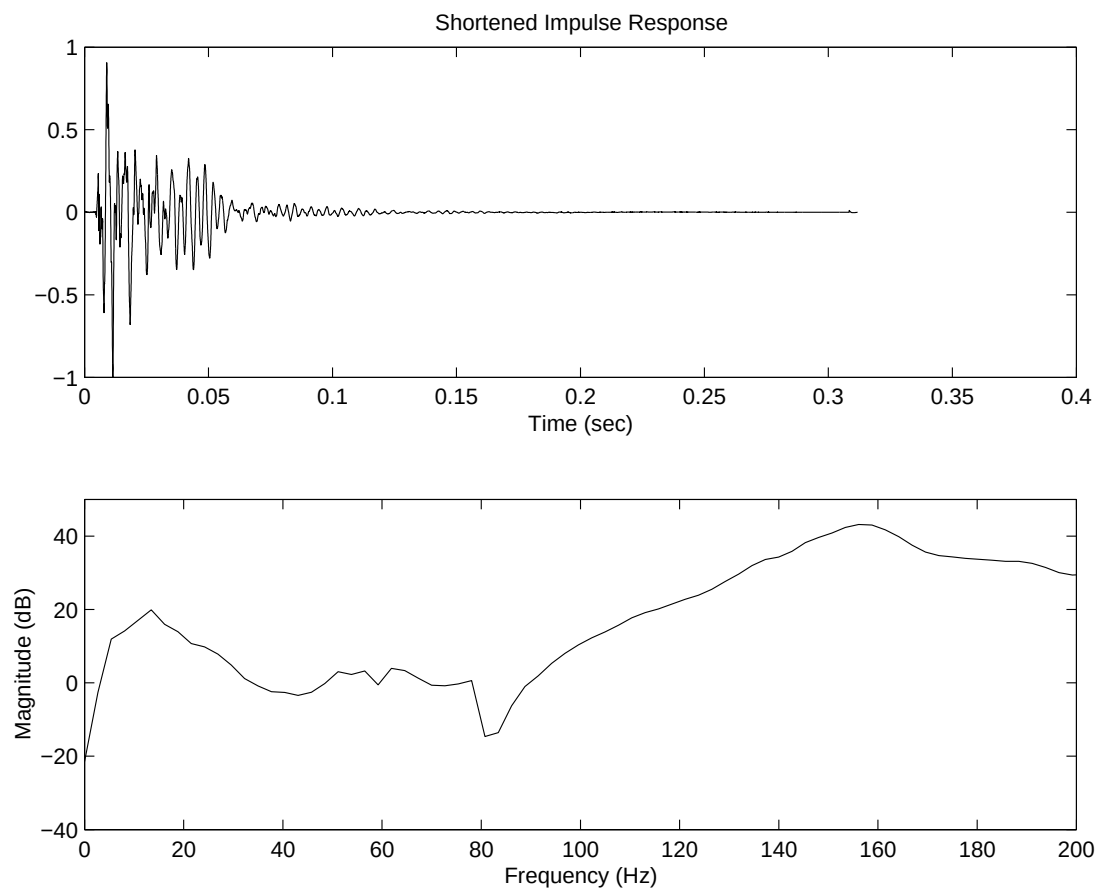


Figure 3: Shown are the impulse response and low-frequency transfer function of a cello body impulse response with modes at 87 and 80 Hz removed. Compared to figure 2, the response is considerably shorter.

The delay lines are divided up so a bow-string junction can be inserted at the point where the bow lies on the string. N_1 and N_2 are chosen according to this bow position. The mechanisms of this junction are described in section 5.

At the bridge, a one-pole, two-zero filter is used. The two zeros model the lossy characteristics of the string while the pole is a simple model of the bridge admittance. The nut (or finger) termination, is assumed to be rigid, thus the travelling velocity waves used are simply inverted at the junction.

4.1 Stiffness

To account for the stiffness of a string, a spatial fourth derivative is added to equation 1 to become the dispersive string equation:

$$Ky'' - \kappa y'''' = \epsilon \ddot{y}. \quad (2)$$

The result of this equation is a stretching of the overtones f_k as

$$f_k = kf_0 \sqrt{1 + kBf_0^2}, k = 1, 2, 3, \dots$$

where B is the inharmonicity factor. For a cello D string, $B = 0.0004Nm^2$ [SSW].

As we did in the homework, the phase delay at each overtone can be computed as

$$P(\omega_k) = \frac{1 - \sqrt{1 + Bk^2}}{f_0 \sqrt{1 + Bk^2}}$$

To simulate these delays, an allpass filter can be used with its poles and zeros optimized to closest fit the desired delays. For this, it happens to be convenient to look at group delay:

$$D(\omega) = -\frac{d\angle H(e^{j\omega})}{d\omega} = P(\omega) + \omega \frac{dP(\omega)}{d\omega}.$$

To compute $D(\omega)$, $P(\omega_k)$ are already determined. An approximate numerical derivative of $P(\omega_k)$ must be taken to get an approximate group delay. At this point, the Matlab function `iirgrpdelay` in the *Filter Design Toolbox 2* can be used. This function creates an “optimal” allpass filter of a specified order to given phase delay samples. Figure 4 shows the ideal group delay of the dispersion of a cello D string (147 Hz, $B = 0.0004Nm^2$) and the group delay of an 16 order allpass filter created using this function. The fit is very good, and I could probably use a smaller order filter to fit this phase delay.

This filter could be placed at any point in the velocity path, and is shown in figure 1 at the nut. The stiffness only significantly affects the transverse waves, thus no filter is included for torsional waves.

5 Bowing Models

There are multiple bowing models that could be used. They range from simple to complex, all with the goal of establishing Helmholtz motion. This occurs when the period of the stick/slip pattern of the bow motion corresponds to the fundamental frequency of the string, and produces a *good* tone.

All the bow-string models are non-linear, and their ultimate goal is to achieve Helmholtz motion at the the bow-string interface. This motion corresponds to the bow sticking to the string and then slipping on the string with a period equal to the fundamental frequency of the string.

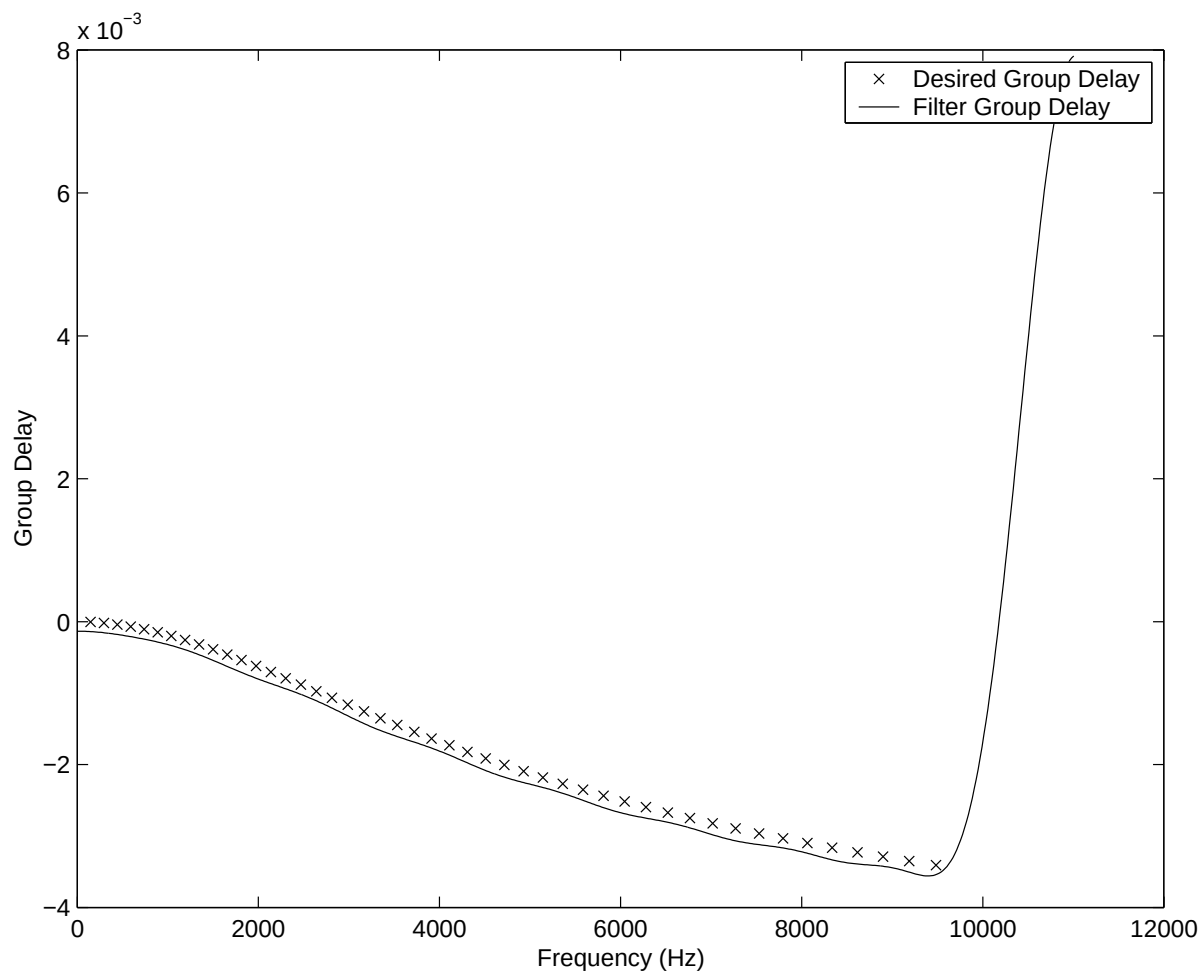


Figure 4: Desired group delay in samples of a stiff cello D string (147 Hz), and the group delay of an 26-order allpass filter created using the Matlab function `iirgrpdelay`

5.1 STK model

The model used in the STK Bowed class computes the friction as

$$f = |(\Delta V + \Delta v_0) * m + 0.75|^{-4}$$

where ΔV is the difference between the string and bow velocity, Δv_0 is some offset, and m is a slope. $f\Delta V$ is added to the positive and negative velocity waves.

5.2 Basic Hyperbolic Model

The model implemented in [SS] calculates the friction force f based on a hyperbolic model:

$$f = \mu(v - v_b)$$

during the stick periods and

$$f = 2Z(v - v_h)$$

during the slip periods. v_b is the bow velocity, v_h is the string velocity, μ is an effective friction coefficient, and Z is the wave impedance of the string. The velocity v is based whether or not the bow is in a stick or slip period, as well as the bow force and the velocities.

This friction force is divided among the travelling velocity waves, adding a $f/(2Z)$ to both directions passing through the junction.

5.3 Modeling Bow Width

To model a finite bow width, a friction force is computed on both edges of the bow. On each edge, the string velocity is computed as the velocity coming towards that edge plus h times the velocity coming towards the other edge, and vice versa [S]. In other words, the string velocity at the left is $v_{hl} = v^+ + hv^-$ and at the right $v_{hr} = h * v^+ + v^-$. For modeling two hairs, one on each edge of the finite bow width, $h = 2$, and two friction forces - f_l and f_r - are computed. The output velocities are $v_{out}^+ = v^+ + f_r/(2Z)$ and $v_{out}^- = v^- + f_l/(2Z)$.

6 Torsional Waves

Torsional waves can be modelled as another string in parallel with the transverse wave string [S]. The speed of torsional waves is 5.2 times greater than that of transverse waves [SSW], making the delays 5.2 times shorter. The other effect of the torsional waves is that they will alter the bow-string interaction. One simple way they can be accounted for is just to modify the impedance in the friction computation model to be

$$Z' = Z \parallel Z_t$$

where Z is the transverse wave impedance and Z_t is the torsional wave impedance.

The friction force then divides up into the transverse and torsion strings according to their individual impedances. For impedance values I used a transversal impedance of 0.55kg/s and a torsional impedance of 1.8kg/s, measured from a cello D string [SSW].

7 Vibrato

Vibrato is implemented by sinusoidally modulating the delay from the bow to the nut. This produces a shift in frequency similar to the Doppler effect. Both the gain and speed of this oscillation are easily controlled.

8 Results

The entire model is implemented in C and is built upon the STK Bowed class. It gave me the basic framework for the string delays and a simple Bow-String junction described in section 5.1.

I have posted a smattering of sample sound files created by various combinations of algorithms on a webpage at <http://www-ccrma.stanford.edu/ɸlarson/cello.html>. Please have a listen and see what you think.

8.1 Bow Models

From just playing around with each of the various bow models, I've made a few observations. They tend to vary in speed of attack (how soon the sound becomes stable), ease of finding usable bow parameters, and, obviously, quality of sound.

The STK Bowed class is the most straightforward of them all. It has fewer parameters, and thus is easiest to pick an appropriate range. It has a decent speed of attack, but I also find it's sound to have the least color.

The hyperbolic model introduced a degree of control over the bow velocity not in the generic STK class. This allows for a little broader variety of sounds, although not all are desirable.

One of the most significant improvements I found was implementing finite bow width. This gives the output a little more graininess and roughness that is heard from a real bow. The models without this give a sound that is a little too uniform, especially if a note is held. Having a small degree of imperfection/randomness introduced by the bow width makes the sound much more real.

I did not get around to plotting the Helmholtz motion, but it is audibly obvious when there is no Helmholtz motion. Excessive slipping sounds just like the bow is slipping and doesn't have a good grip on the string. I cannot distinguish the other variations in motion as easily, but I'm sure I've heard them.

8.2 Torsional Waves

Implementing the torsional waves was one of the trickier tasks. The range of bow velocity and bow force inputs that produce a good tone with a clean attack is not very large. Many combinations produce a lot of slipping on the attack, and others simply produce no sound or higher harmonics. I also found that tweaking the pole location of the torsional wave bridge filter gave a better sound.

8.3 String Stiffness

The impact of the dispersion filter that accounts for string stiffness is most evident in the more stripped down models. The dispersion gives a very nice color to the sound, giving a brighter tone. The difference with and without the dispersion filter is quite noticeable.

8.4 Body

I originally was going to use two Bi-Quads at 80 and 87 Hz to simulate the removed modes of the cello body impulse response. Yet I found that using both of these at similar, low frequencies gave the sound a very heavy tone with a little too much bass. Removing one of the Bi-Quads fixed this up and made the sound a bit nicer.

8.5 Bowings

I had originally intended to try and think about ways to improve some of the bow phrasing, thinking about slurring notes, string crossings, staccato, etc. Yet I found that, with a decent model in place, many of these phrasings come naturally. For example, staccato bowing can be done by short, hard (high force) strikes of the string with the bow. The staccato sample posted was created using short duration, high bow force pulses and then releasing. It sounds decent. I think the SKINI-file interpreter I'm using stops taking input after NoteOff's, without which the ring of the string would be more sustained.

9 Conclusions

The end result of this project is quite pleasing and effective at modeling a cello sound. The process of trying various additions to the model step by step helped me to realize what how the different parts of the model affect the sound. It has also been fun to play around with various types of sounds and include some real music.

I would like to try a few more things with this model. One would be to implement the "Plastic" bow model described in [PW]. This accounts for rosin heating up and the resulting hysteresis of friction. I just have run out time to really debug. I also want to play with some of the controls and create a more expressive piece with my model.

Using the physical model as a guide, it is possible to build a very good virtual stringed instrument. The most challenging aspects are building a good string stiffness model and having an effective bow-string interaction relationship. Even with these more complex components, reasonably simple models still perform very well, as is evidenced by the output of this virtual cello.

References

- [S] Serafin, Stefania. "Impact of String Stiffness on Virtual Bowed Strings", CCRMA DSP Seminar Presentation, Music 423, Stanford University, May 2002.
- [G] Guettler, Knut. "The Bowed String: On the Development of Helmholtz Motion and On the Creation of Anomalous Low Frequencies," Doctorate Thesis, Royal Institute of Technology - Speech, Music and Hearing, Stockholm, April 5, 2002.
- [SSW] Serafin, Stefania, Julius O. Smith III, and Jim Woodhouse. "An Investigation on the Impact of Torsion Waves and Friction Characteristics on the Playability of Virtual Bowed Strings," Proceedings of the IEEE Workshop on Applications of Signal Processing to Audio and Acoustics, New Paltz, New York, Oct. 17-20, 1999.
- [SS] Serafin, Stefania and Julius Smith. "Bowed String Synthesis Example," Tutorial on Virtual Musical Instruments, Acoustical Society of America Newport Beach Meeting, Dec. 4, 2000.
- [PW] Pitteroff R, Woodhouse J. "Mechanics of the contact area between a violin bow and a string Part I: reflection and transmission behaviour", *Acustica/Acta Acustica*, 84, 543-562, (1998).

Pitteroff R, Woodhouse J. "Mechanics of the contact area between a violin bow and a string Part II: simulating the bowed string", *Acustica/Acta Acustica*,84, 744-757, (1998).

Pitteroff R, Woodhouse J. "Mechanics of the contact area between a violin bow and a string Part III: parameter dependence", *Acustica/Acta Acustica*,84, 929-946, (1998).